

Prompt 1 - Define User Stories

- **Purpose of AI Use:** Brainstorming and requirements to start project
- **AI-Suggested Content:**
- **Validation:** Read through and found some User Stories are too broad in the feature, and more detail is needed to be added and priorities to be moved around.
- **Decision:** Partially Accepted
- **Reflection:** AI helped give a starting point for the project, and to start spreading the work with the team
- **Responsible Person:** Paul Fourment

Prompt

Here is a project given, I will be working with a team, repo has been setup and a basic svelte kit app has been created, no database setup yet or anything else started. Per the given instructions give 10 user stories that we can use to start.

(User Story instructions and Project Description Pasted From PDF)

Response

Here are 10 user stories covering the core CareerConnect features. The first four are ordered so your Sprint 1 demo (Registration/Login + Profile/Resume) is covered. Each follows the Appendix B format, with acceptance criteria and a starting task breakdown you can turn into GitHub Issues.

Summary

ID	Title	Primary User	Priority	Sprint
US-01	Account registration	Job Seeker / Recruiter	High	1
US-02	Login and logout	Job Seeker / Recruiter	High	1
US-03	Profile management	Job Seeker	High	1
US-04	Resume upload and management	Job Seeker	High	1
US-05	Job posting management	Recruiter	High	TBD
US-06	Job search and filtering	Job Seeker	High	TBD

US-07	Job application submission	Job Seeker	High	TBD
US-08	Application status tracking	Recruiter	Medium	TBD
US-09	Application history dashboard	Job Seeker	Medium	TBD
US-10	Saved jobs	Job Seeker	Medium	TBD

US-01: Account registration

Priority: High · Sprint: 1

As a job seeker, I want to create an account with my email and a password so that I can access the platform's features.

Acceptance criteria

- ☐ Email must be unique and valid.
- ☐ Password meets minimum strength rules.
- ☐ The user chooses a role (Job Seeker or Recruiter).
- ☐ A clear error message is shown for invalid input.
- ☐ After signup, the user is logged in or redirected to the login page.

Tasks

- **Task 1.1:** Choose the database/auth approach and design the users schema (id, email, password hash, role, created_at).
 - **Task 1.2:** Implement the signup endpoint/form action with server-side validation and password hashing.
 - **Task 1.3:** Build the registration page UI in SvelteKit with client-side validation.
 - **Task 1.4:** Write tests for valid signup, duplicate email, and weak password.
-

US-02: Login and logout

Priority: High · Sprint: 1

As a registered user, I want to log in and log out securely so that my personal data stays protected.

Acceptance criteria

- ☐ Correct credentials create a session.
- ☐ Wrong credentials show a generic error message.
- ☐ Protected routes redirect unauthenticated users to the login page.
- ☐ Logout ends the session.

Tasks

- Task 2.1: Implement session handling (e.g. in `hooks.server.ts`).
 - Task 2.2: Build the login page and logout action.
 - Task 2.3: Add route guards for authenticated pages and role-based pages.
 - Task 2.4: Write tests for login success, login failure, and protected-route redirects.
-

US-03: Profile management

Priority: High · Sprint: 1

As a job seeker, I want to create and edit my profile (name, location, skills, education, experience summary) so that recruiters and the platform can understand my background.

Acceptance criteria

- ☐ The user can view and edit profile fields.
- ☐ Changes persist after a page refresh.
- ☐ Required fields are validated.
- ☐ Users can only edit their own profile.

Tasks

- Task 3.1: Design the profiles schema linked to users.
 - Task 3.2: Implement load and update logic for the profile page.
 - Task 3.3: Build the profile view/edit UI.
 - Task 3.4: Test authorization so one user cannot modify another user's profile.
-

US-04: Resume upload and management

Priority: High · Sprint: 1

As a job seeker, I want to upload, view, replace, and delete my resume so that I can use it when applying to jobs.

Acceptance criteria

- ☐ Only PDF (and optionally DOCX) files under a size limit are accepted.
- ☐ The uploaded resume can be downloaded or previewed.
- ☐ The user can replace or delete the resume.
- ☐ Files are only accessible to the owner (and to recruiters for jobs the user applied to).

Tasks

- Task 4.1: Choose file storage (cloud bucket, or local storage for development).
 - Task 4.2: Design the resumes schema (file path, file name, upload date, owner).
 - Task 4.3: Implement upload/delete endpoints with file type and size validation.
 - Task 4.4: Build the resume management UI.
 - Task 4.5: Test invalid file types, oversized files, and access control.
-

US-05: Job posting management

Priority: High

As a recruiter, I want to create, edit, and close job postings so that job seekers can find and apply to my openings.

Acceptance criteria

- ☐ Postings include title, company, location, job type, description, requirements, and application deadline.
- ☐ Only recruiters can create postings.
- ☐ Recruiters can only edit their own postings.
- ☐ Closed postings no longer accept applications.

Tasks

- Task 5.1: Design the jobs schema.
- Task 5.2: Implement create/update/close actions with role checks.
- Task 5.3: Build the recruiter posting form and "My Postings" page.

US-06: Job search and filtering

Priority: High

As a job seeker, I want to search jobs by keyword and filter by location, job type, and posting date so that I can quickly find relevant opportunities.

Acceptance criteria

- ☐ Keyword search matches job title, company, and description.
- ☐ Filters can be combined.
- ☐ Results show key details and link to a job detail page.
- ☐ An empty state is shown when there are no results.

Tasks

- **Task 6.1:** Implement the search xquery with filters and pagination.
 - **Task 6.2:** Build the search page with filter controls and URL query parameters.
 - **Task 6.3:** Build the job detail page.
-

US-07: Job application submission

Priority: High

As a job seeker, I want to apply to a job using one of my uploaded resumes and an optional cover note so that I can submit applications directly through the platform.

Acceptance criteria

- ☐ The user selects a resume before applying.
- ☐ Duplicate applications to the same job are prevented.
- ☐ The application is recorded with status "Applied".
- ☐ The recruiter can see the application.

Tasks

- **Task 7.1:** Design the applications schema (job, applicant, resume, status, timestamps).
 - **Task 7.2:** Implement the apply action with duplicate prevention.
 - **Task 7.3:** Build the apply flow on the job detail page.
 - **Task 7.4:** Build the recruiter view of applicants for a posting.
-

US-08: Application status tracking

Priority: Medium

As a recruiter, I want to update an application's status (Applied, Interview, Offered, Rejected) so

that applicants know where they stand.

Acceptance criteria

- ☐ Only the posting's recruiter can change the status.
- ☐ Status changes are timestamped.
- ☐ The applicant sees the updated status.

Tasks

- **Task 8.1:** Implement the status update action with authorization.
 - **Task 8.2:** Store status history (status + changed_at).
 - **Task 8.3:** Add status controls to the recruiter applicant view.
-

US-09: Application history dashboard

Priority: Medium

As a job seeker, I want a dashboard listing all my applications with their current status so that I can stay organized.

Acceptance criteria

- ☐ Applications show job title, company, date applied, and current status.
- ☐ The list can be filtered by status.
- ☐ Summary counts per status are shown.

Tasks

- **Task 9.1:** Implement a dashboard data loader.
 - **Task 9.2:** Build the dashboard UI with status filters and summary counts.
-

US-10: Saved jobs

Priority: Medium

As a job seeker, I want to save jobs to a favourites list so that I can review and apply to them later.

Acceptance criteria

- ☐ The user can save and unsave jobs from search results and the job detail page.
- ☐ Saved jobs appear on a dedicated page.
- ☐ Closed postings are clearly marked.

Tasks

- Task 10.1: Design the saved_jobs schema.
- Task 10.2: Implement save/unsave actions.
- Task 10.3: Build the Saved Jobs page and save toggle component.

Not Covered (left for team brainstorming)

- Deadline notifications and reminders
- The required Generative AI feature
- The team's original feature